



DevSaini2512 add

d239a28 · 3 minutes ago



82 lines (67 loc) · 2.63 KB

Preview

Code

Blame



Raw



check file permission

code

```
1  #!/bin/bash
2  # check_file.sh
3  # Usage: ./check_file.sh filename.txt
4
5  if [ $# -ne 1 ]; then
6      echo "Usage: $0 <filename>"
7      exit 1
8  fi
9
10 file="$1"
11 if [ -e "$file" ]; then
12     echo "File exists: $file"
13     echo "----- contents -----"
14     cat -- "$file"
15 else
16     echo "File '$file' does not exist."
17     read -p "Create it now? (y/N): " ans
18     case "$ans" in
19         [Yy]*) touch "$file"; echo "Created $file"; echo "You can edit it using your favorite editor." ;;
20         *) echo "Not creating file." ;;
21     esac
22 fi
```

line by line explanation

```
#!/bin/bash
```



This line tells the system to use the Bash shell to run the script.

```
# check_file.sh
# Usage: ./check_file.sh filename.txt
```



explanation:

These are comments explaining what the script is and how to run it.

```
if [ $# -ne 1 ];
```

**explanation :**

then — Start an if that checks the number of positional arguments: \$

```
#
```



is the count of arguments;



means “not equal to 1”. If the script was not given exactly one argument, the then block runs.

```
echo "Usage: $0 "
```

**explanation :**

— Print a usage message. \$0 expands to the script name (how it was invoked).

- exit 1 — Exit the script with status 1 (conventional non-zero exit code means an error).
- fi — End of the if block.
- file="\$1" — Save the first positional argument (\$1) into the variable file. Quotes around "\$1" preserve spaces and special characters.
- if [-e "\$file"]; then — Test whether the path in \$file exists (-e is true for files, directories, symlinks, etc.). If it exists, run the then block. (Use -f if you specifically want a regular file.)
- echo "File exists: \$file" — Inform the user that the file exists.
- echo "----- contents -----" — A visual separator before printing the file contents.

```
cat -- "$file"
```



- Print the file contents to stdout. The -- stops option parsing (so filenames starting with - won't be treated as options); quoting "\$file" protects spaces/newlines in the filename.
- else — Otherwise (the file does not exist), run the following block.
- echo "File '\$file' does not exist." — Inform the user the file was not found.
- read -p "Create it now? (y/N): " ans — Prompt the user and read their answer into variable ans. -p supplies the prompt text. The prompt shows (y/ N) indicating the default/expected choice is No.
- case "\$ans" in — Start a case statement to branch based on the user's response (pattern matching).
- [Yy]) touch "\$file"; echo "Created \$file"; echo "You can edit it using your favorite editor." ;; — If the answer begins with Y or y ([Yy] matches Y, y, Yes, yep, etc.), run the commands separated by ;; touch creates an empty file (or updates its timestamp), then print confirmation messages. ;; ends this case branch.
-) echo "Not creating file." ;; — Default branch (matches anything else): print that the file will not be created.
- esac — End the case statement.
- fi — End the outer if (the if [-e ...] block).

output:

```
devsaini@ubuntu:~/Documents/linux_assingment/Assingment/Lab7.md$ ./check_file.sh Armstrong.c
File exists: Armstrong.c
----- contents -----
#include <stdio.h>

int main(){
    int num,sum,rem,temp;

    printf("Armstrong numbers between 1 and 500 are:\n");

    for(num = 1; num <=500; num++)
    {
        sum=0;
        temp=num;

        for(; temp>0; temp /= 10)
        {
            rem = temp % 10;
            sum += rem * rem * rem;
        }

        if(sum == num) {
            printf("%d\n",num);
        }
    }
    return 0;
}
```

